

VHDL Fundamentals

Module Convenor: Dr. T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

Department of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus

Objectives

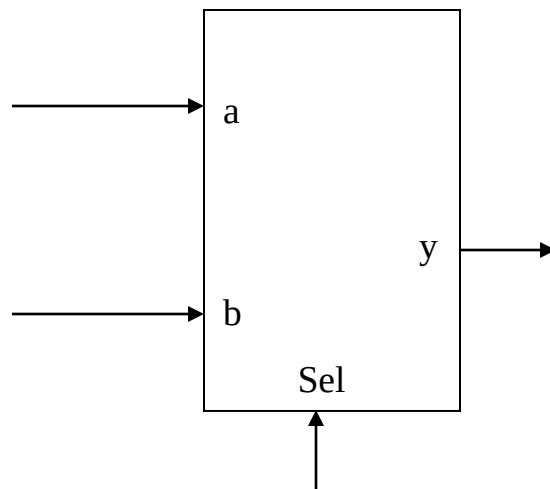
After completing this lecture, you will be able to:

- Understand the different design units in VHDL
- Understand the Behavioural and Structural VHDL coding
- To be able to identify the simple VHDL coding errors
- To be able to write a Behavioural and Structural VHDL code for a simple combinational logic.

What exactly is VHDL?

A way of describing the operation of a digital system

Example: a two input multiplexer



```
entity mux2 is
    Port ( a : in std_logic;
           b : in std_logic;
           sel : in std_logic;
           y : out std_logic);
end mux2;

architecture Behavioral of mux2 is
begin
    if sel = 1 then
        y <= a;
    else
        y <= b;
    end if;
end Behavioral;
```

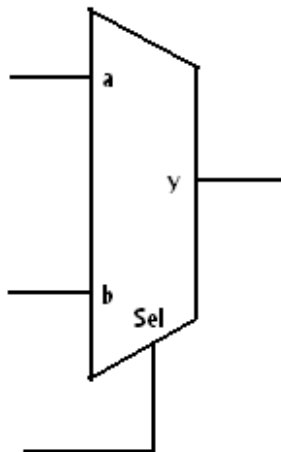
VHDL Design Units

- VHDL has 5 different ‘design units’
- A ‘design unit’ consists of VHDL textual code
- The 5 VHDL ‘design units’ are:
 - **Entities** (essential)
 - **Architectures** (essential)
 - **Packages** (optional)
 - **Package Bodies** (optional)
 - **Configurations** (optional)
- These five ‘Design units’ can be compiled separately and stored in a library or WORK area
- We shall concentrate for the moment on the **essential** units, namely:
 - **Entities**
 - **Architectures**

Entity

- ◆ Ports declaration for Inputs and outputs
- ◆ Upper most level of the design
- ◆ Defines the interface between entity and the environment

Two Input Mux



```
-- The Entity of Two input Mux
|
entity mux2 is
    Port ( a : in std_logic;
          b : in std_logic;
          sel : in std_logic;
          y : out std_logic);
end mux2;
```

Entity (cont...)

Syntax

Entity *<identifier_name>* is

Port (*<identifier>* : *<mode>* *<data type>* ;
 <identifier> : *<mode>* *<data type>*) ;

End *<identifiers name>* ;

```
-- The Entity of Two input Mux
|
entity mux2 is
    Port ( a : in std_logic;
           b : in std_logic;
           sel : in std_logic;
           y : out std_logic );
end mux2;
```

Note:

Reserved words are in Blue

Comments are in Green and start with '--' until end of line

Code is case insensitive ie Rdy=rdy=RDy

statements terminated by ';' (may run over a line)

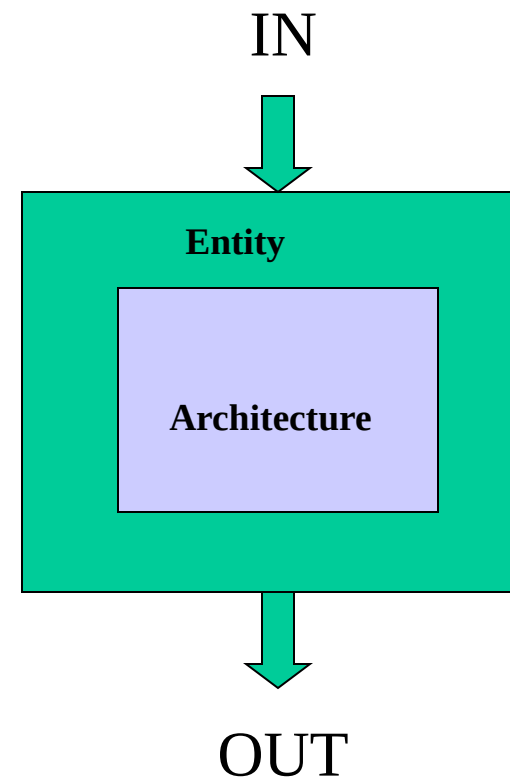
list delimiter ','

- An architecture body describes relationship between the design entity inputs and the outputs
- Every architecture should have an entity
- The architecture name shouldn't be same as the entity name
- Two types
 - Behavioural
 - Structural
- An entity can be linked with several architecture
- The names of entity and architecture should not reflect the names of library functions (or VHDL Keywords)

Architecture (cont...)

```
entity mux2 is
    Port ( a : in std_logic;
           b : in std_logic;
           sel : in std_logic;
           y : out std_logic);
end mux2;

architecture Behavioral of mux2 is
    -- signal and variable declaration
begin
    -- statement ie relationship between I/O s
end Behavioral;
```



Architecture (cont...)

Behavioural Architecture

```
entity mux2 is
    Port ( a : in std_logic;
          b : in std_logic;
          sel : in std_logic;
          y : out std_logic);
end mux2;

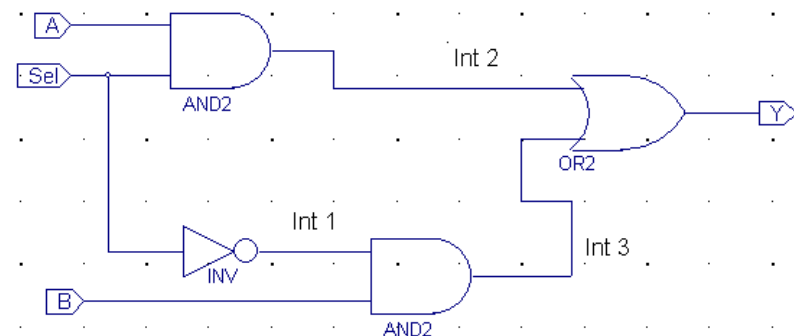
architecture Behavioral of mux2 is

begin
    if sel = 1 then
        y <= a;
    else
        y <= b;
    end if;
end Behavioral;
```

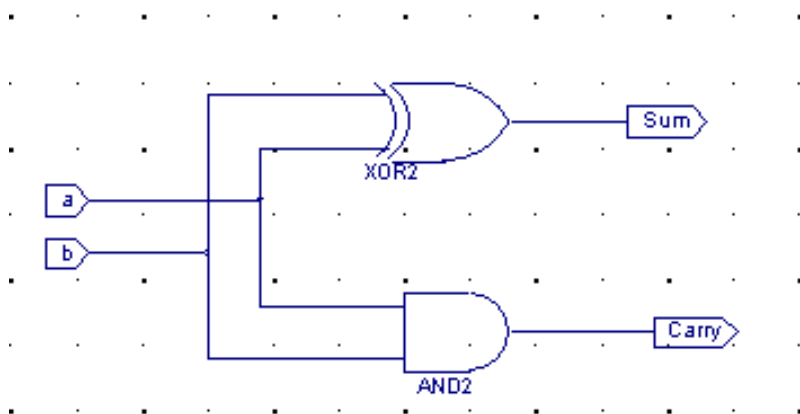
Syntax

```
architecture <architecture name> of
    <entity_identifier> is
    [ <architecture_declarative_part> ]
begin
    <architecture_statement_part>

end [<architecture_name>];
```



Write the Behavioral VHDL code for a Half Adder Circuit

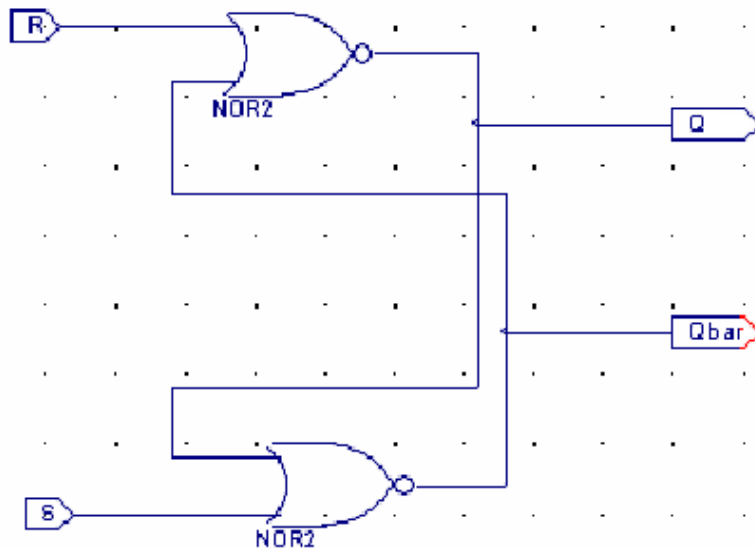


```

halfadder *
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  Entity ha is
7      port ( a, b : in std_logic;
8              sum, carry : out std_logic);
9  end ha;
10 Architecture ha_behav of ha is
11     begin
12         sum <= a xor b;
13         carry <= a and b;
14     end ha_behav;
15
16

```

Write the Behavioral VHDL code for a RS Flip Flop



```

16  Library ieee;
17  Use ieee.std_logic_1164.all;
18
19  Entity rsff is
20    port ( s , r : in std_logic;
21           q , qbar :out std_logic);
22  end rsff ;
23
24  Architecture rsff_behav of rsff is
25    signal x,y :std_logic;
26    begin
27      y <= s nor x;
28      x <= r nor y ;
29      q <= x ;
30      qbar <= y;
31  end rsff_behav;
  
```

Mode

- **in** — component only can read the signal
- **out** — component only write to the signal
- **inout** — component can read or write to the signal (bi directional)
- **buffer** — component can write and read back the signal (no bi directional)
- **linkage** — used only for the documentation

Data Type

Boolean type - True or false

Integer type - Integer (32 bit) **(-2147483648 to +2147483648)**

- binary [B"10101"]

- Octal [O"721"]

- Hexadecimal [X"AF14"]

Character type - 0 to 9, A to Z, a to z, special characters(+,-,*,/,
etc.,)

Bit type - 0,1,X,Z

Entity (cont...)

Std_logic

U – un initialized

X - forced don't care (forced un known or contention)

W – weak unknown

1 - forced one

0 - forced zero

L - weak zero

H - weak one

Z - high impedance

'-- ' - don't care

Eg: signal B : in std_logic;

B <= '0';

Note: use single quotes to represent a bit

Data Type

Array/Vector Type : ‘to’ – ascending order

‘downto’ – descending order

Example:

```
A : in std_logic_vector (9 downto 0);
```

```
A <= “110111001”
```

```
[A(9) = 1, A(8) = 1, A(7) = 0.....A(0) = 1]
```

Note : Use double quotes for a vector

```
B : out std_logic_vector (0 to 3);
```

```
[B(0), B(1),.....B(3)]
```

It is designers choice to use “downto” or “to”, but mix use of both is discouraged

Eg:

```
Signal A: std_logic_vector (4 downto 1);
```

```
Signal B: std_logic_vector (1 to 4);
```

```
A <= B; is equivalent to ????????????????
```

Exercise 1

- Assume the below assigning condition,
 A : in `std_logic_vector` (4 *downto* 0);
 B : in `std_logic_vector` (0 *to* 4);
If $A \leq "10010"$ & $B \leq A$, then determine
the value of B(4).

Slice of Array

Assign value to a bit or part of a array

B(0) <= '0'; B(2 to 3) <= "10"; Also A(1) <= B(0);

Note: Slice direction must be the same as in the declaration

Time 1ms, 9ns, 100ps

Enumerated Type:

TYPE color IS (red, blue, green, orange);

```
type stateType is (stIdle, stData, stStop, stTxdCompleted);
```

```
attribute enum_encoding of statetype: type is "00 01 11 10";
```

```
signal presState: stateType;
```

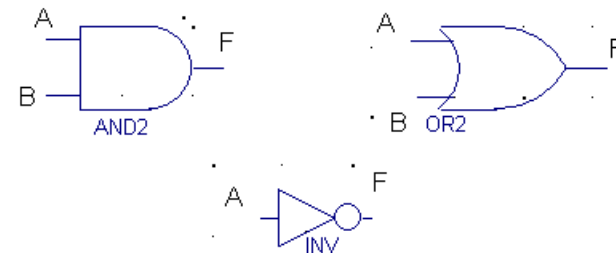
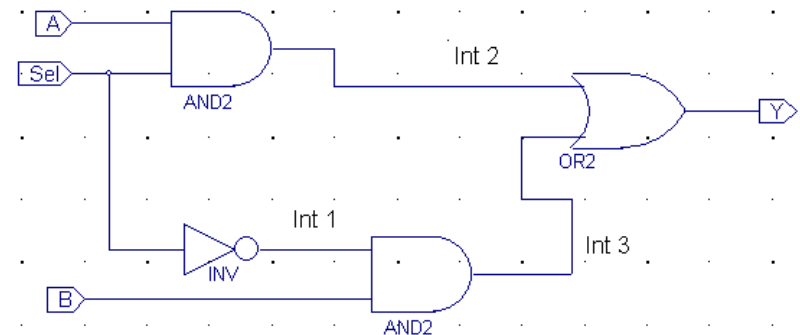
```
signal nextState: stateType;
```

VHDL Keywords

abs	component	guarded	next	range	subtype
access	configuration	if	nor	record	then
after	constant	impure	not	register	to
alias	disconnect	in	null	reject	transport
all	downto	inertial	of	rem	type
and	else	inout	on	report	unaffected
architecture	elsif	is	open	rol	units
array	end	label	or	ror	until
assert	entity	library	others	select	use
attribute	exit	linkage	out	severity	variable
begin	file	literal	package	signal	wait
block	for	loop	port	shared	when
body	function	map	postponed	sla	while
buffer	generate	mod	procedure	sll	with
bus	generic	nand	process	sra	xnor
case	group	new	pure	srl	xor

Structural Architecture

- Inter connection of number of components
- Synthesizable code
- Used for Hierarchical VHDL design
- Hierarchical design is usually used for a design more than 1000 gates
- uses component declaration to declare components
- signals to connect the entity ports to the components



Structural Architecture

```
entity mux2 is
    Port ( a : in std_logic;
           b : in std_logic;
           sel : in std_logic;
           y : out std_logic);
end mux2;

architecture Structure of mux2 is
    -- local components declaration
    component AND2 is
        port ( a,b : in std_logic; f: out std_logic);
    end component;

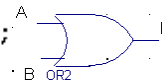
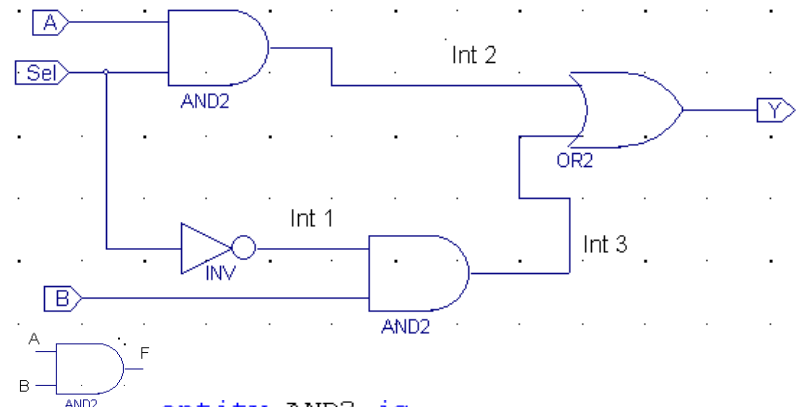
    component OR2 is
        port (a,b : in std_logic; f: out std_logic);
    end component OR2;

    component INV is
        port (a : in std_logic; f: out std_logic);
    end component;

    signal int1, int2, int3 : std_logic;
begin

    U1: INV    port map (sel, int1);
    U2: AND2   port map (sel,a,int2);
    U3: AND2   port map (int1,b,int3);
    U4: OR2    port map (int2,int3,y);

end Structure;
```



```
entity AND2 is
    port (A, B : in std_logic;
          F: out std_logic);
end AND2;
Architecture component_AND2 of AND2 is

    F <= A AND B;
end component_AND2;

entity OR2 is
    port (A,B : in std_logic;
          F : out std_logic);
end OR2;
Architecture component_OR2 of OR2 is

    F <= A OR B;
end component_OR2;

entity inv is
    port ( A : in std_logic;
          F : out std_logic);
end inv;
Architecture component_inv of inv is
    F <= NOT A;
end component_inv;
```

Architecture (cont...)

- Component Declaration

- Declaration part goes between architecture and begin

Syntax:

```
component <name>
    [ generic ( <generic associated list>);]
    port (<port associated list>);
end component <name>;
```

- Port Map

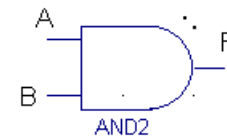
- It is used to link other components to an instanced component

Syntax

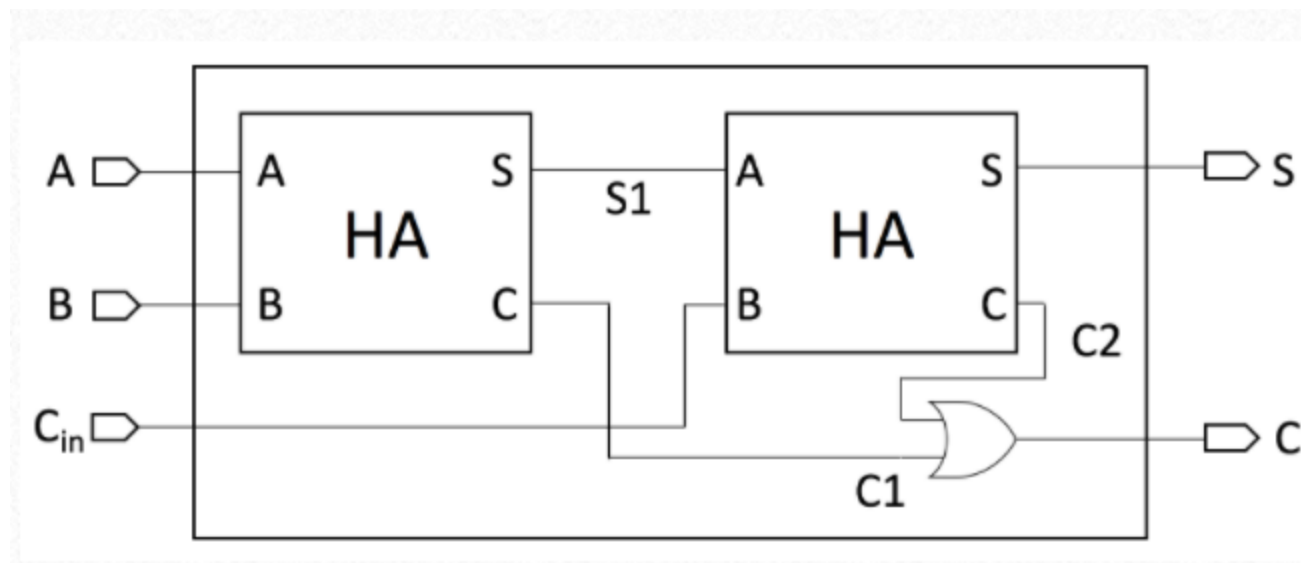
```
port map (<port-association-list>);
```

```
U2: AND2    port map (sel,a,int2);
```

```
U2: AND2 port map (a => sel, b => a, f=> int2);
```



Hierarchical design of a full adder circuit



VHDL DESIGN

```
entity full_adder is
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          c : in  STD_LOGIC;
          sum : out STD_LOGIC;
          carry : out STD_LOGIC);
end full_adder;

architecture Behavioral of full_adder is

component half_adder is
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          sum : out STD_LOGIC;
          carry : out STD_LOGIC);
end half_adder;
signal int_sum, int_carry1, int_carry2 :std_logic;
begin
    u1: half_adder port map (
        a      => a,
        b      => b,
        sum    => int_sum,
        carry  => int_carry1);

    u2: half_adder port map (
        a      => int_sum,
        b      => c,
        sum    => sum,
        carry  => int_carry2);

    carry <= int_carry1 and int_carry2;

end Behavioral;
```

```
entity half_adder is
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          sum : out STD_LOGIC;
          carry : out STD_LOGIC);
end half_adder;

architecture Behavioral of half_adder is

begin
    sum <= a xor b;
    carry <= a and b;

end Behavioral;
```

VHDL std_logic_1164

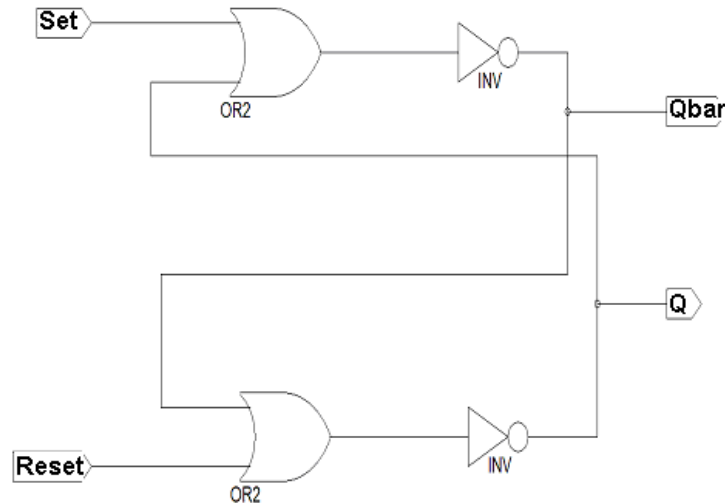
Use of std_logic_1164

```
library IEEE;                                -- library declaration
use IEEE.STD_LOGIC_1164.ALL;                 -- package declaration
use IEEE.STD_LOGIC_ARITH.ALL;                -- package declaration
use IEEE.STD_LOGIC_UNSIGNED.ALL;            -- package declaration
```

Declare the above library and packages in the beginning of all your codes.

Exercise 2

- Write a structural VHDL code for the schematic diagram shown in Figure



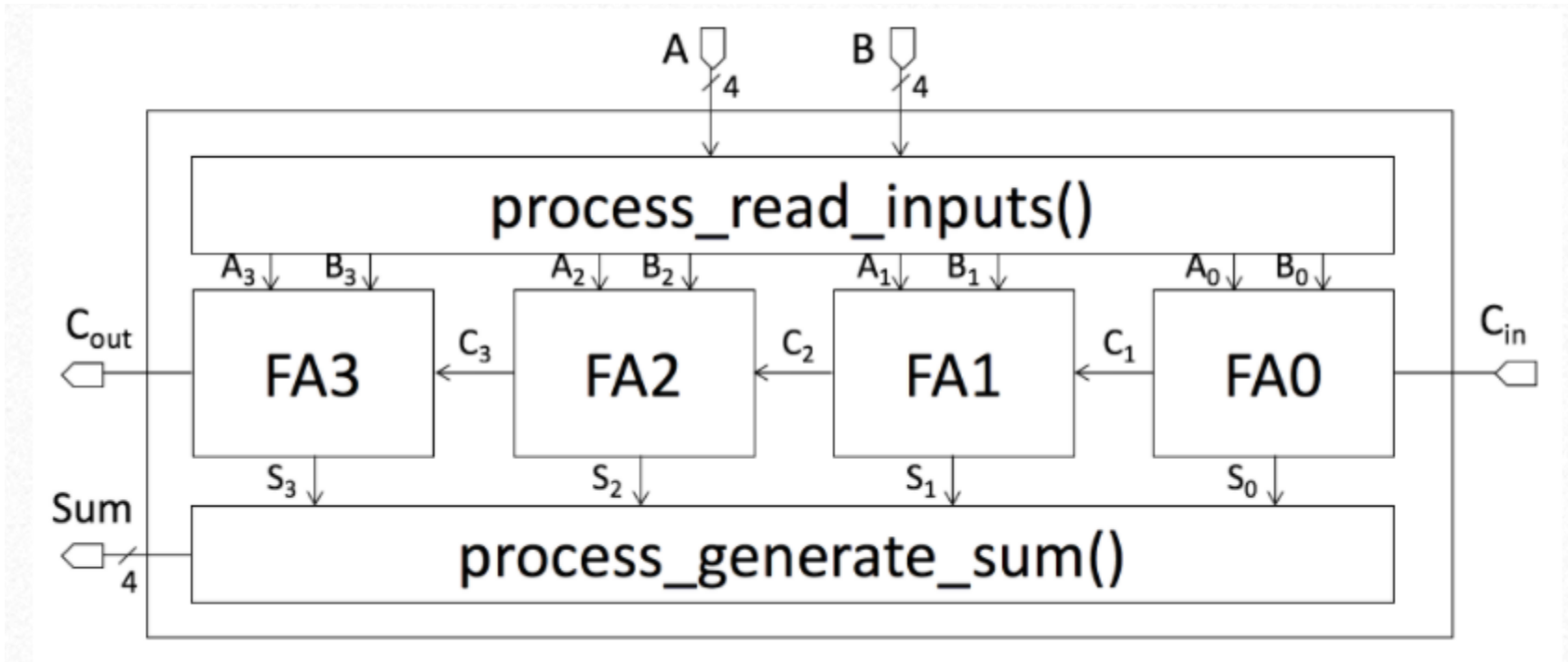
Exercise 3

Identify any TWO errors in the following VHDL code and correct them.

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 entity casewhen_err is
26     Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
27           B : in  STD_LOGIC_VECTOR (1 downto 0);
28           Sum : in  Integer range 0 to 5);
29 end casewhen_err;
30
31 architecture case of casewhen_err is
32
33 begin
34
35     Case A & B is
36         when "00001" => sum <= "001",
37         when "00010" => sum <= "110",
38         when others => sum <= 5;
39     end;
40
41
42
43 end;
```

Exercise 5

Write the hierarchical design of a 4-bits Ripple Carry Adder circuit using four full adders.



Exercise 6

1. What is meant by **mode** in an entity?
2. Name the built-in data types used in an entity?
3. Which of these are valid identifiers:
behav2; myfriend_4; 2go_home; x; all; after_8;
4. Where would you use “**port map**” and where would you use “**port**”?
5. Name the three levels used in an architecture body.
6. Within “**port map**”, what does => signify?
7. What mode will be used if no mode applies in port list?
In; out; inout;

Exercise 7

True/False

1. An entity defines behaviour ?
2. An entity is equivalent to a symbol ?
3. You can place behaviour inside an entity ?
4. An entity must have only **one** architecture ?
5. In VHDL '87 an identifier can begin with a number ?
6. External signals are declared in an architecture body ?
7. Mode is specified on internal signals ?
8. ++ is used to define a comment statement ?
9. Component declaration can be in an entity declaration?
10. Signals can be declared within an entity declaration?



E]

More on syntax



2

`::=` means definition “entity declaration `::=`”
`:=` means default value of the port
`{ }` means optional and repeating
`[ ]` means optional
`|` means or “constant | variable”

